

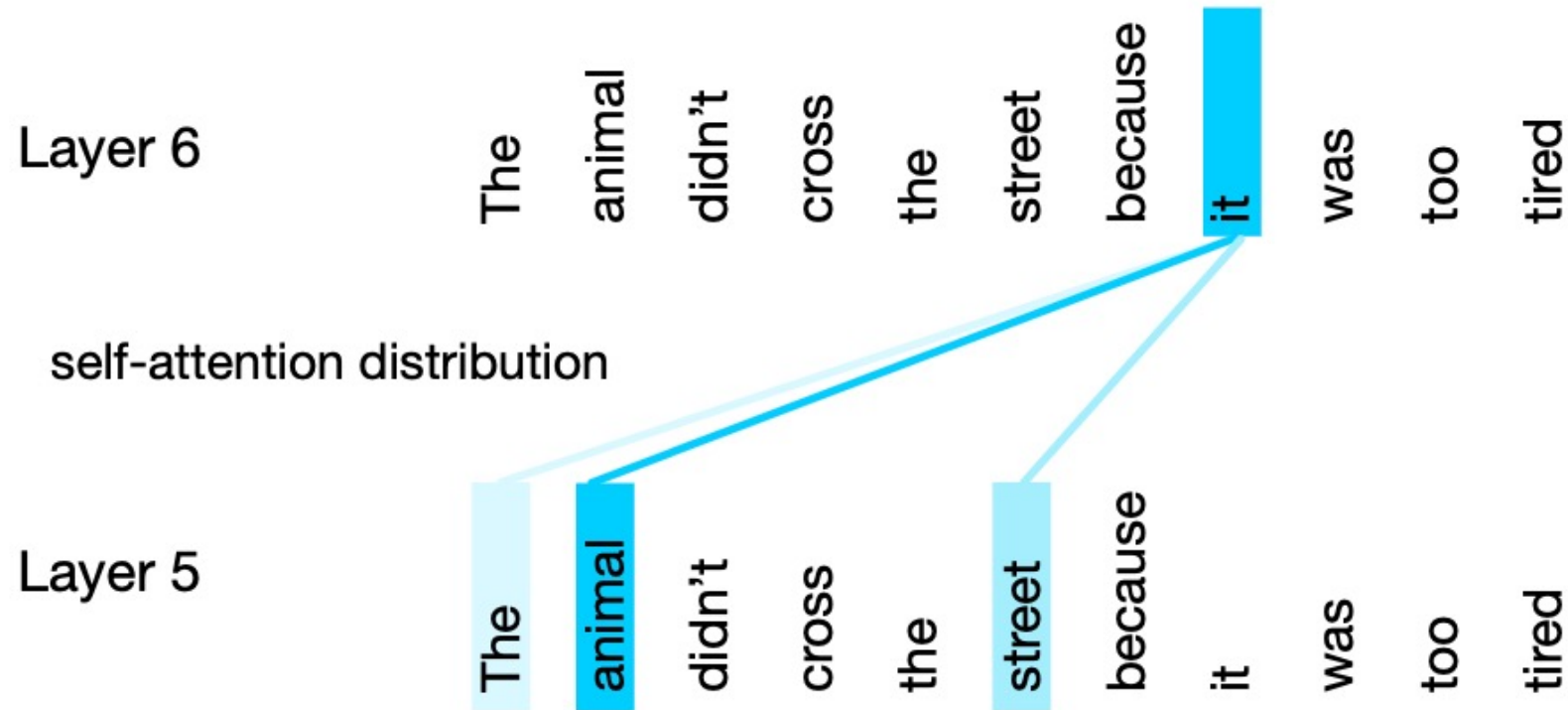
Transformers

C. V Jawahar

IIIT-Hyderabad

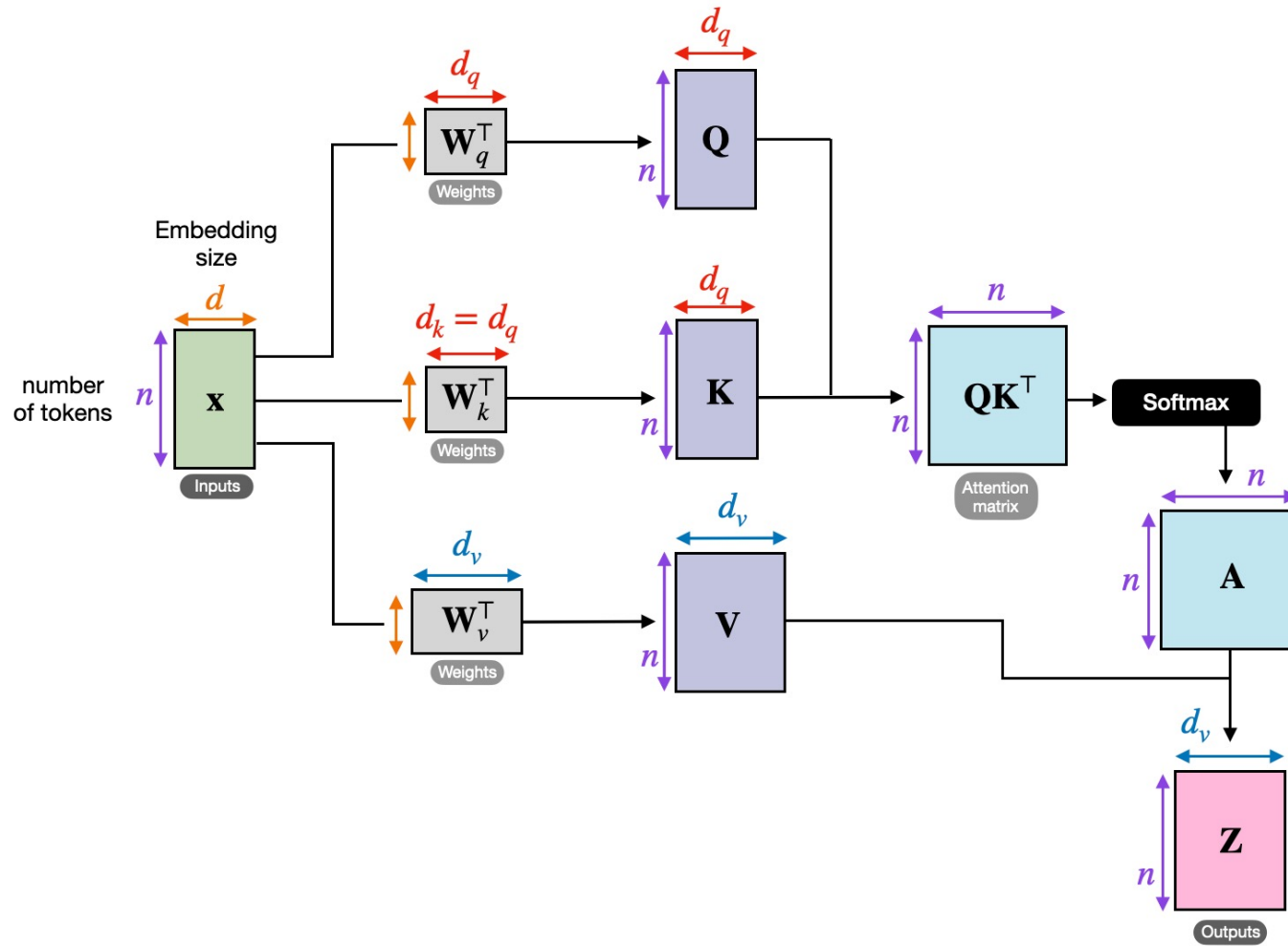
16 April 2026

Self Attention



1. **The animal** did not cross the road because **it** was too tired.
2. The animal did not cross **the road** because **it** was too wide.

Self Attention: Computation



Multi-Head Attention

1) This is our input sentence*

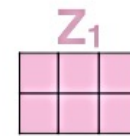
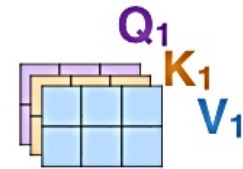
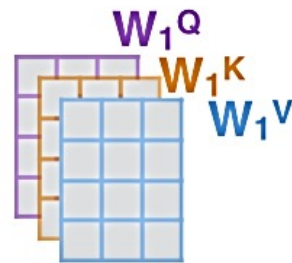
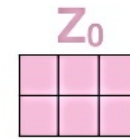
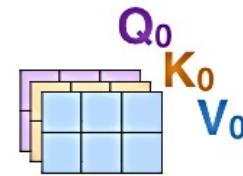
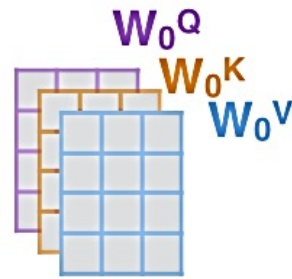
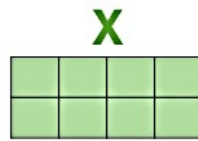
2) We embed each word*

3) Split into 8 heads. We multiply X or R with weight matrices

4) Calculate attention using the resulting $Q/K/V$ matrices

5) Concatenate the resulting Z matrices, then multiply with weight matrix W^O to produce the output of the layer

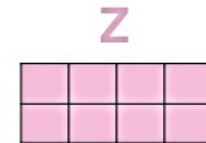
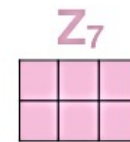
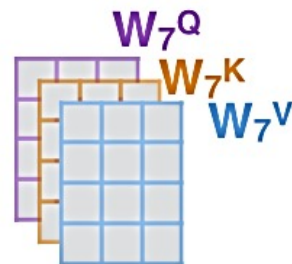
Thinking
Machines



...

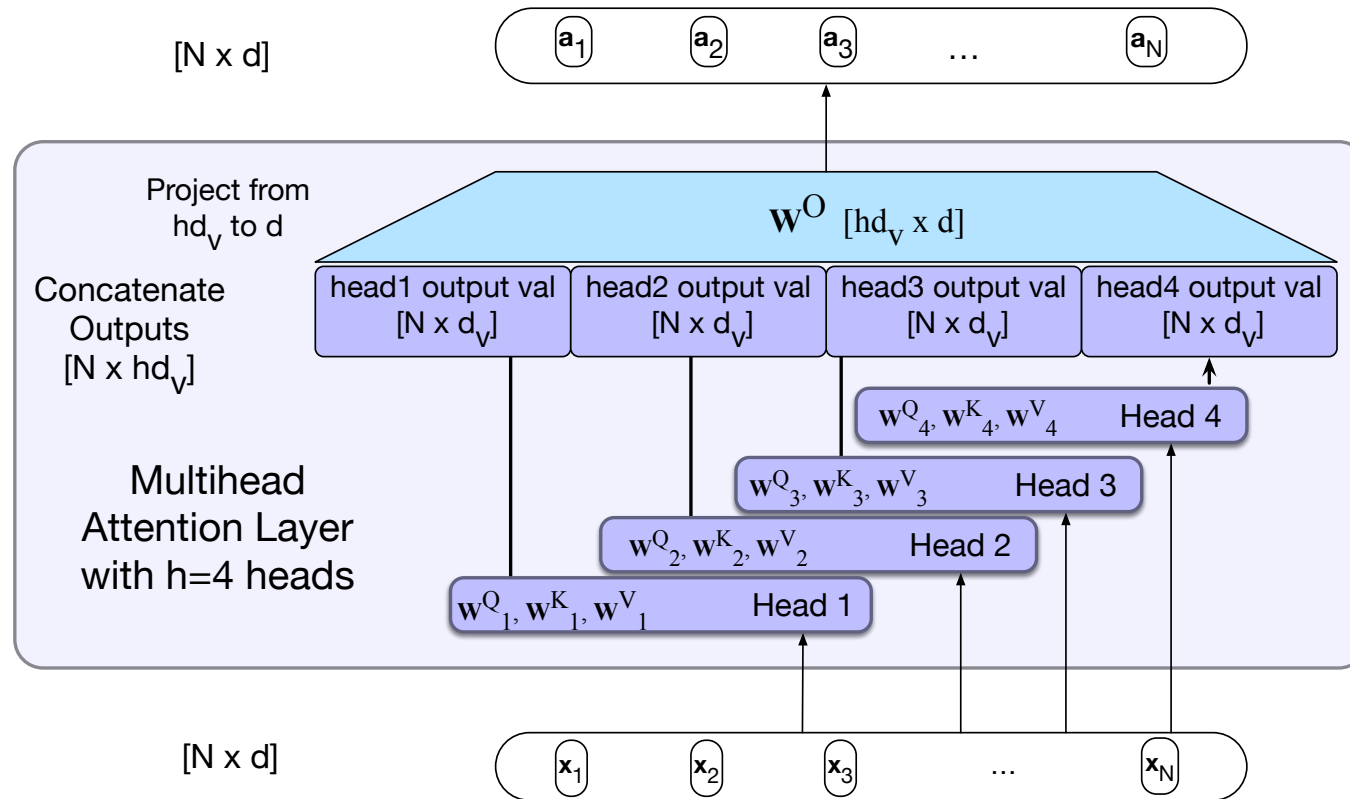
...

...



* In all encoders other than #0, we don't need embedding. We start directly with the output of the encoder right below this one

Summary Multi Head Attention



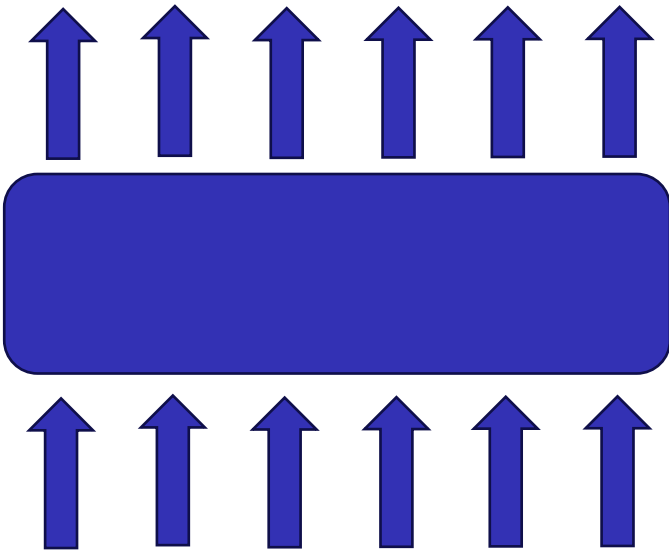
$$\mathbf{Q} = \mathbf{XW}_i^Q ; \mathbf{K} = \mathbf{XW}_i^K ; \mathbf{V} = \mathbf{XW}_i^V$$

$$\text{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\text{head}_1 \oplus \text{head}_2 \dots \oplus \text{head}_h) \mathbf{W}^O$$

MLP: Fully connected layer

Input: N vectors of d dimension



Input: N vectors of d dimension

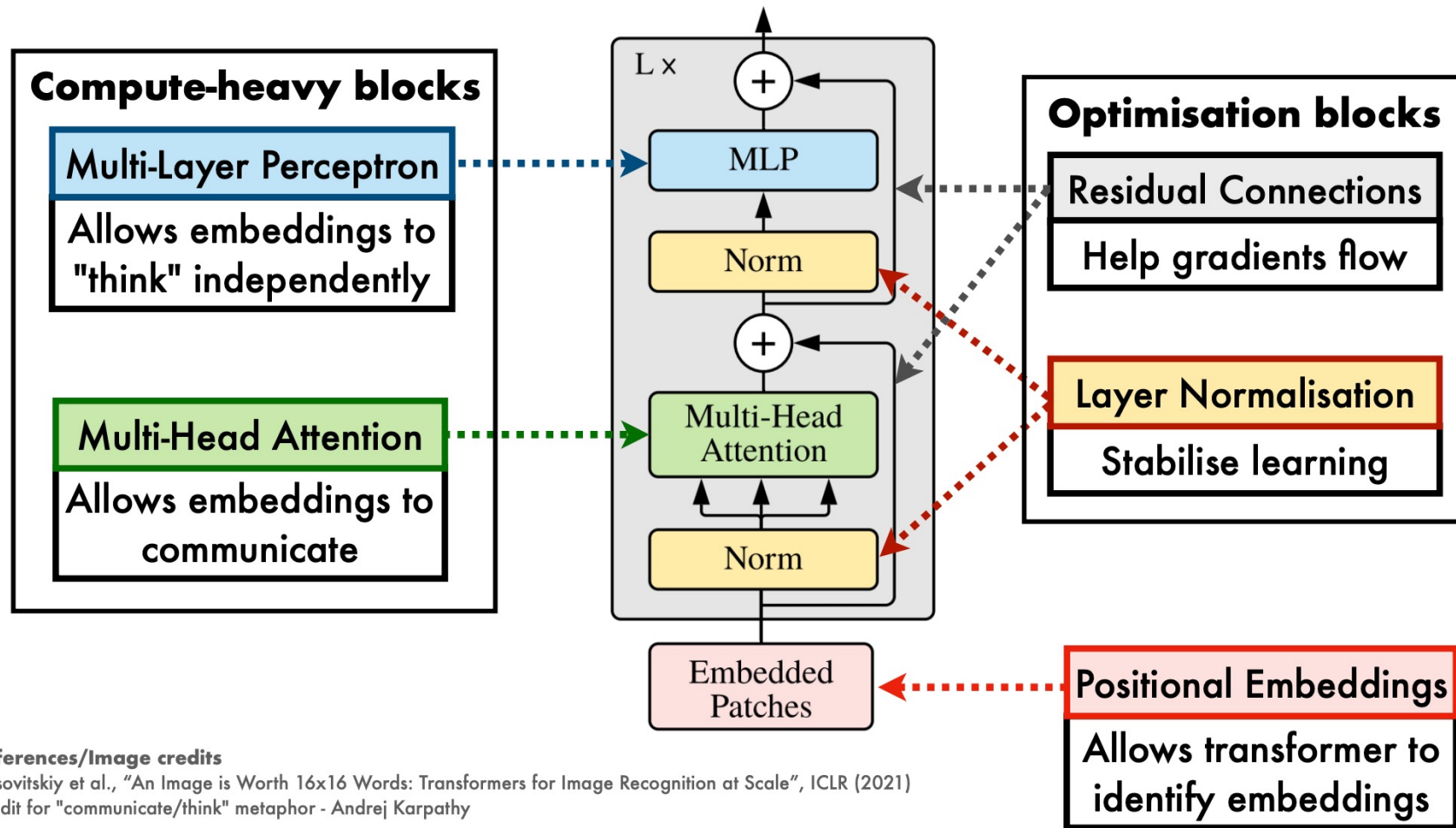
- Let $N=1000$ and $d = 300$

- MLP
 - Fully Connected
 - Number of parameters to learn
 - $(1000 \times 300) \times (1000 \times 300)$
 - $= 9 \times 10^{10}$
- CNN
 - Let $N=1000$ (say sqr) and $d = 300$
 - Number of parameters to learn
 - $300 \text{ (channels)} \times 3 \times 3 \times 300$
 - $= 27 \times 10^4$
- RNN
 - Let $N=1000$ and $d = 300$ (and $h = 100$)
 - Number of parameters to learn
 - $300 \times 100 \text{ (U)}; 100 \times 100 \text{ (W)}; 100 \times 300 \text{ (V)}$
 - $= 7 \times 10^4$
- SA
 - Let $N=1000$ and $d = 300$; Project to h ; $h = 100$
 - $300 \times 100 \text{ (W}^Q\text{)}; 300 \times 100 \text{ (W}^K\text{)}; 300 \times 100 \text{ (W}^V\text{)}; 100 \times 300 \text{ (W}^O\text{)}$
 - $= 12 \times 10^4$

Transformers

Transformer Encoder

Five key ideas

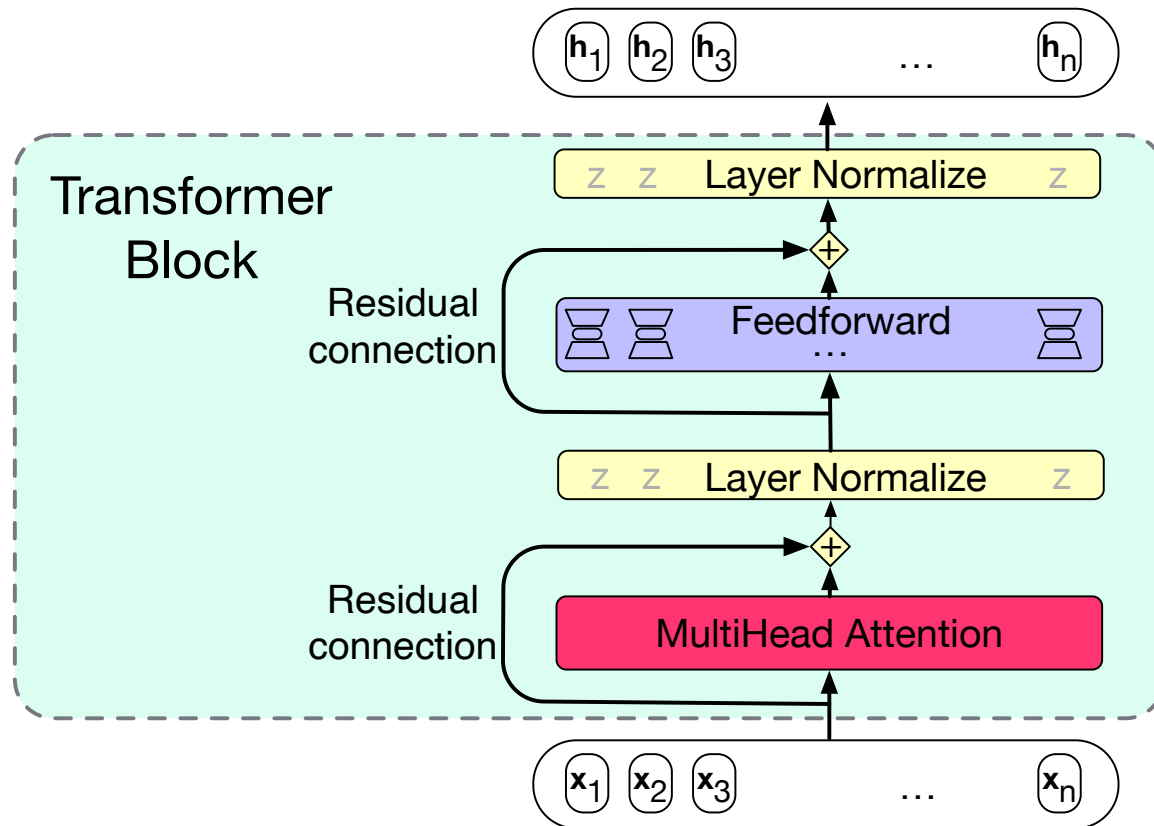


References/Image credits

Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR (2021)

Credit for "communicate/think" metaphor - Andrej Karpathy

Transformer Block



Both X and H are of $N \times d$

$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X})$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2)$$

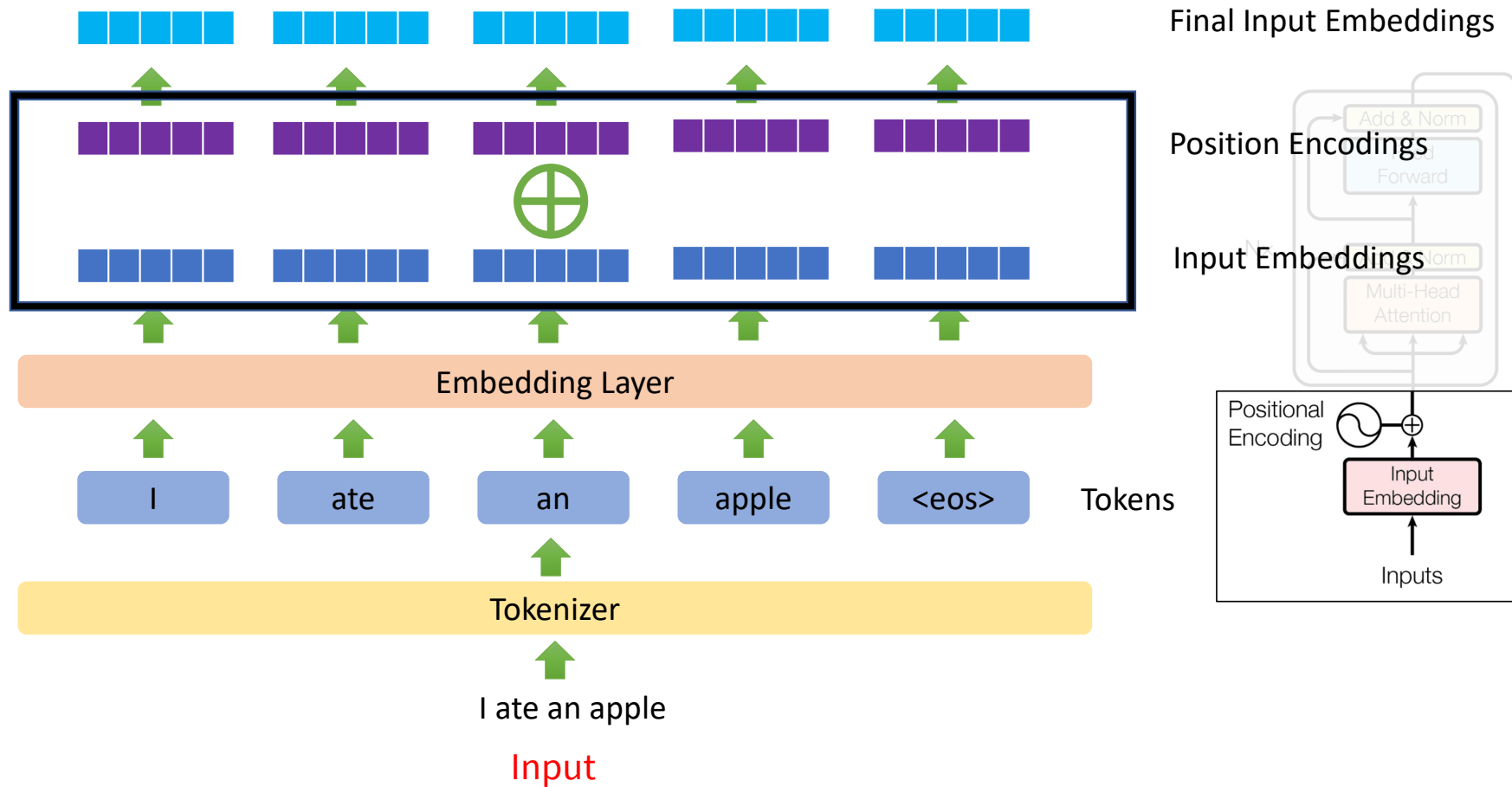
$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3)$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5)$$

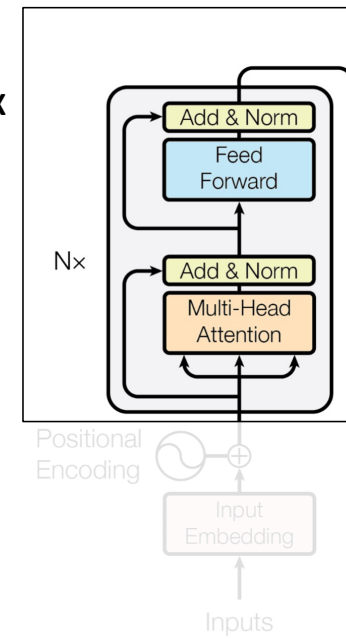
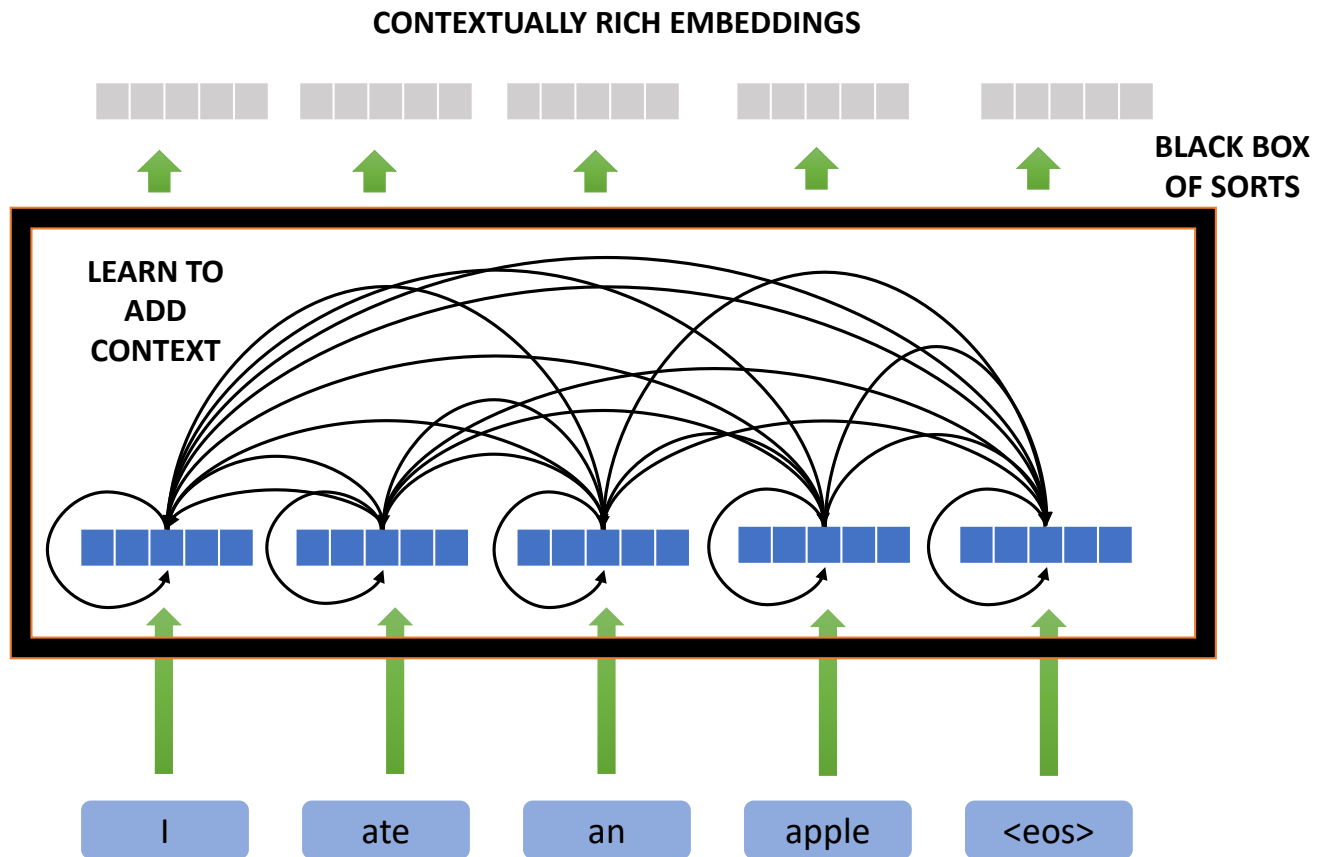
Steps

Positional Encoding

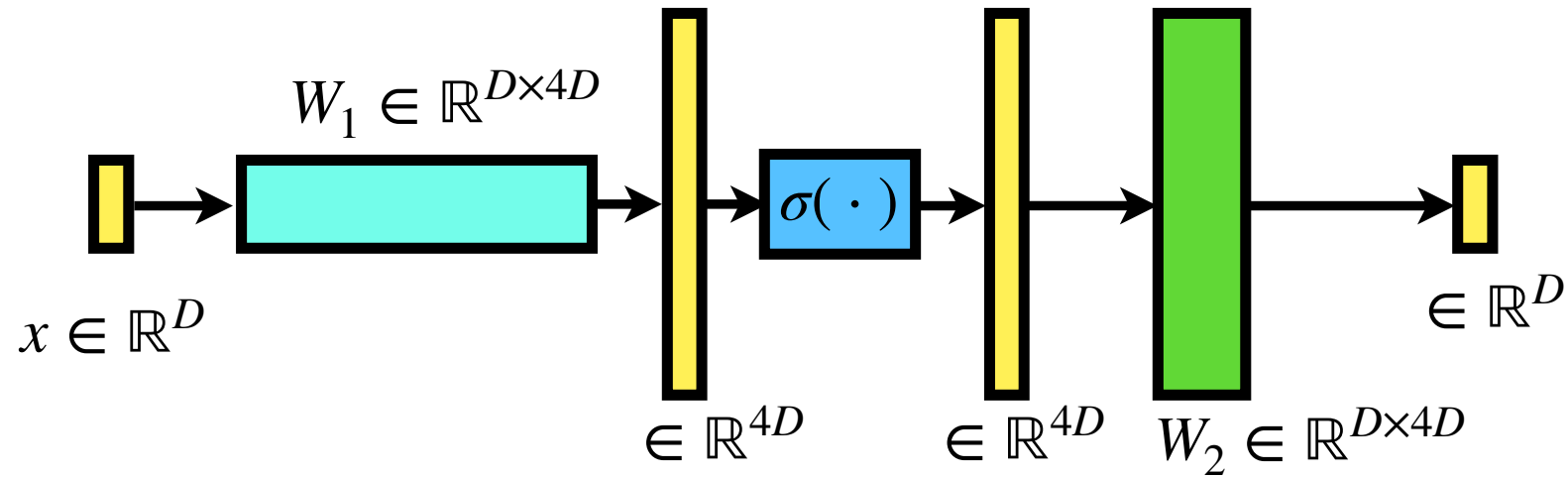


Steps

Encoder



MLP/Feed Forward Layer



$$\text{MLP}(x) = W_2 \sigma(W_1 x + b_1) + b_2$$

Extract combination features from the attended outputs

Layer Norm Enhances the Stability

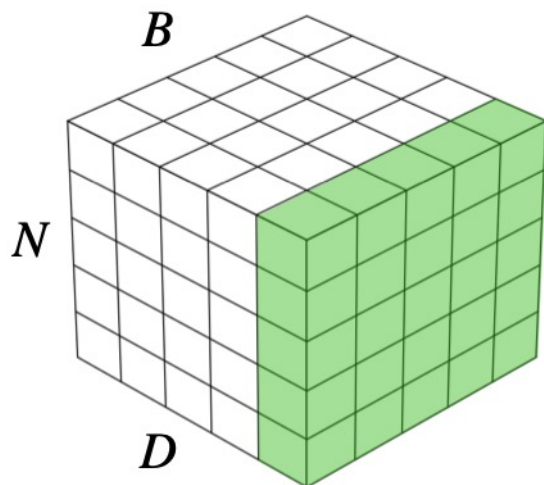
$$\mu = \frac{1}{d_h} \sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h} \sum_{i=1}^{d_h} (x_i - \mu)^2}$$

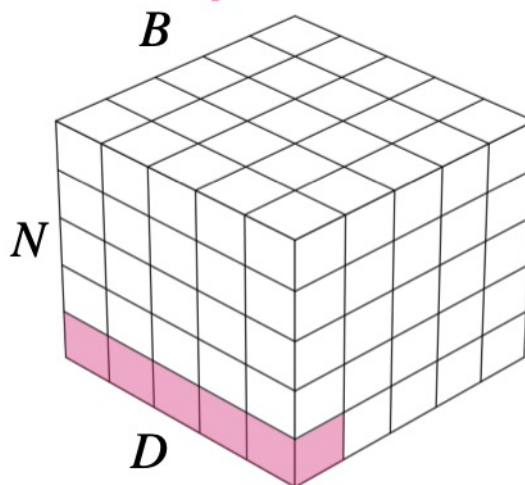
$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm} = \gamma \hat{\mathbf{x}} + \beta$$

BatchNorm



LayerNorm



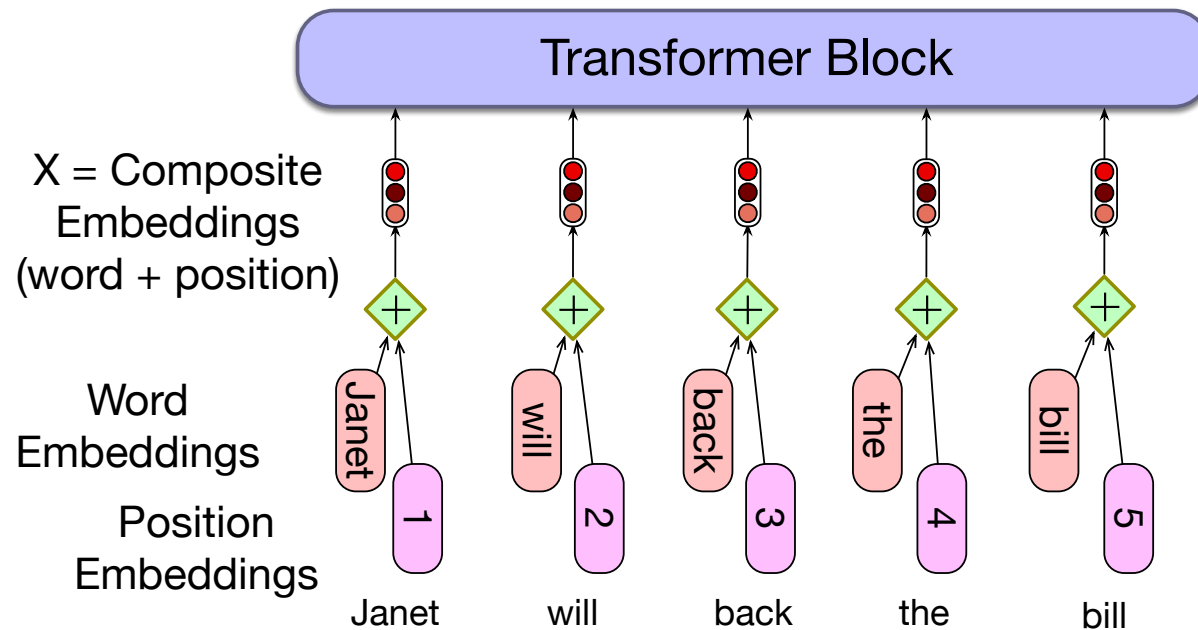
LayerNorm has

- No dependence on batch dim.
- Same procedure at train/test time

Positional Encoding

- Sets vs Sequences=> Strategy: Introduce “Position”
- Position embedding soon after input embedding
- “Bag of words” to “ordered words”

Position Embedding

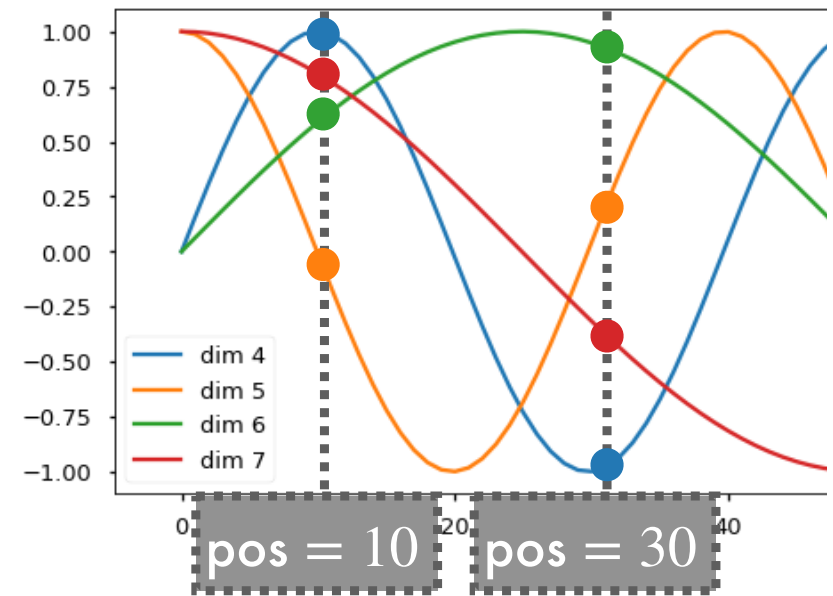


Absolute Positions

Choose static functions (say sinusoids) that map integer inputs to real valued vectors that captures inherent relationship between vectors

$$PE(pos, 2i) = \sin\left(\frac{pos}{10,000^{2i/D}}\right)$$

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10,000^{2i/D}}\right)$$



Many Improvements

- Simplifies LayerNorm by removing the mean and bias terms

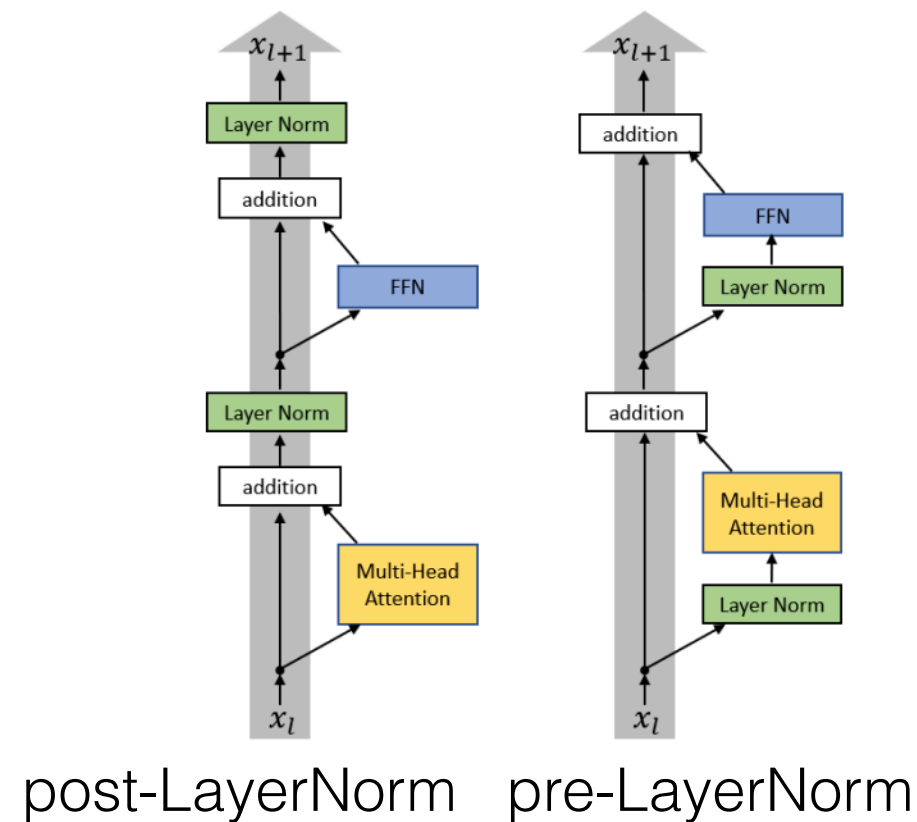
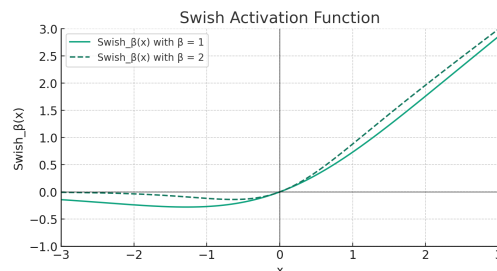
$$\text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2}$$

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \cdot \mathbf{g}$$

RMS Norm

SiLU

$$\text{Swish}(\mathbf{x}; \beta) = \mathbf{x} \odot \sigma(\beta \mathbf{x})$$

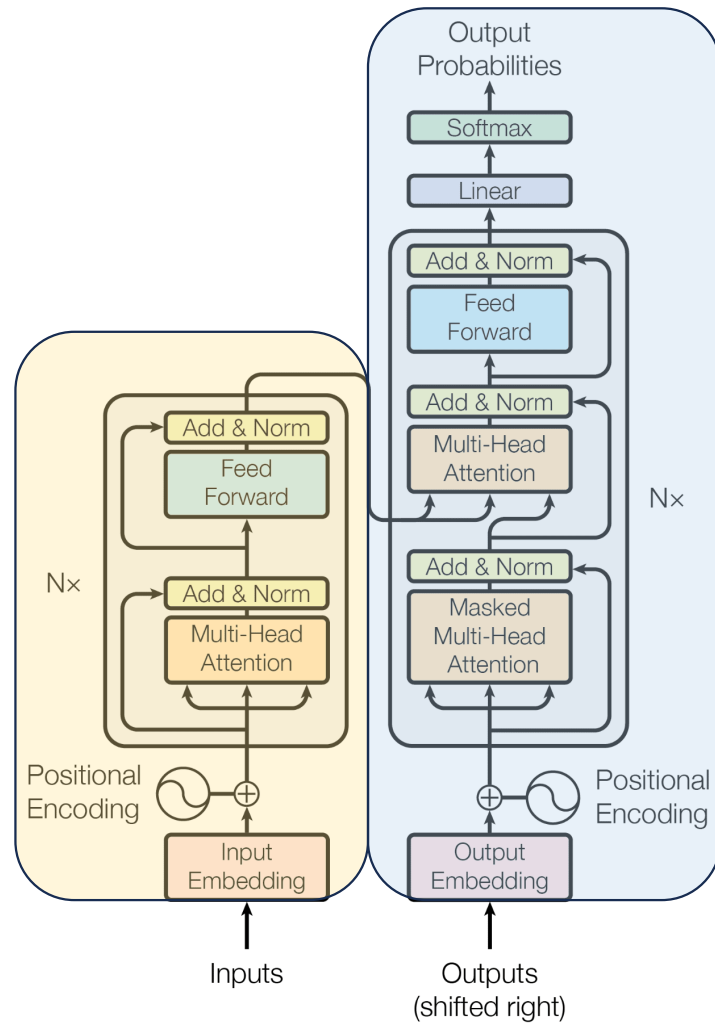


	Vaswani et al.	LLaMA
Norm Position	Post	Pre
Norm Type	LayerNorm	RMSNorm
Non-linearity	ReLU	SiLU
Positional Encoding	Sinusoidal	RoPE

Approximately 10X more efficient (compute vs Performance)

Transformers (2017)

Representation



Generation

Roles

Input – input tokens

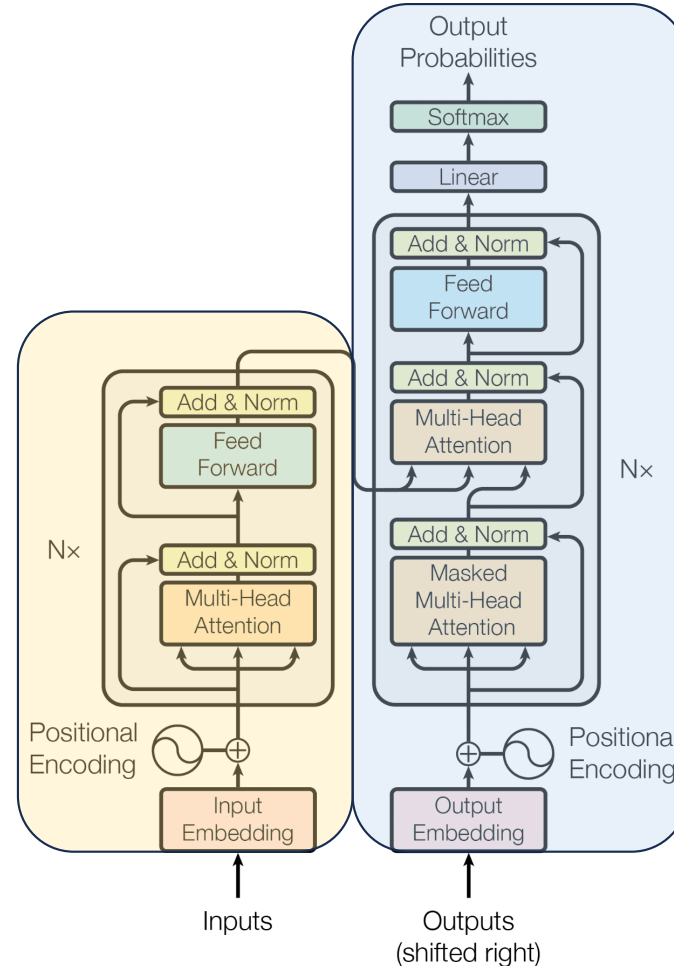
Output – hidden states

Model can see all timesteps

Does not usually output tokens, so no inherent auto-regressivity

Can also be adapted to generate tokens by appending a module that maps hidden state dimensionality to vocab size

Representation



Input – output tokens and hidden states*

Output – output tokens

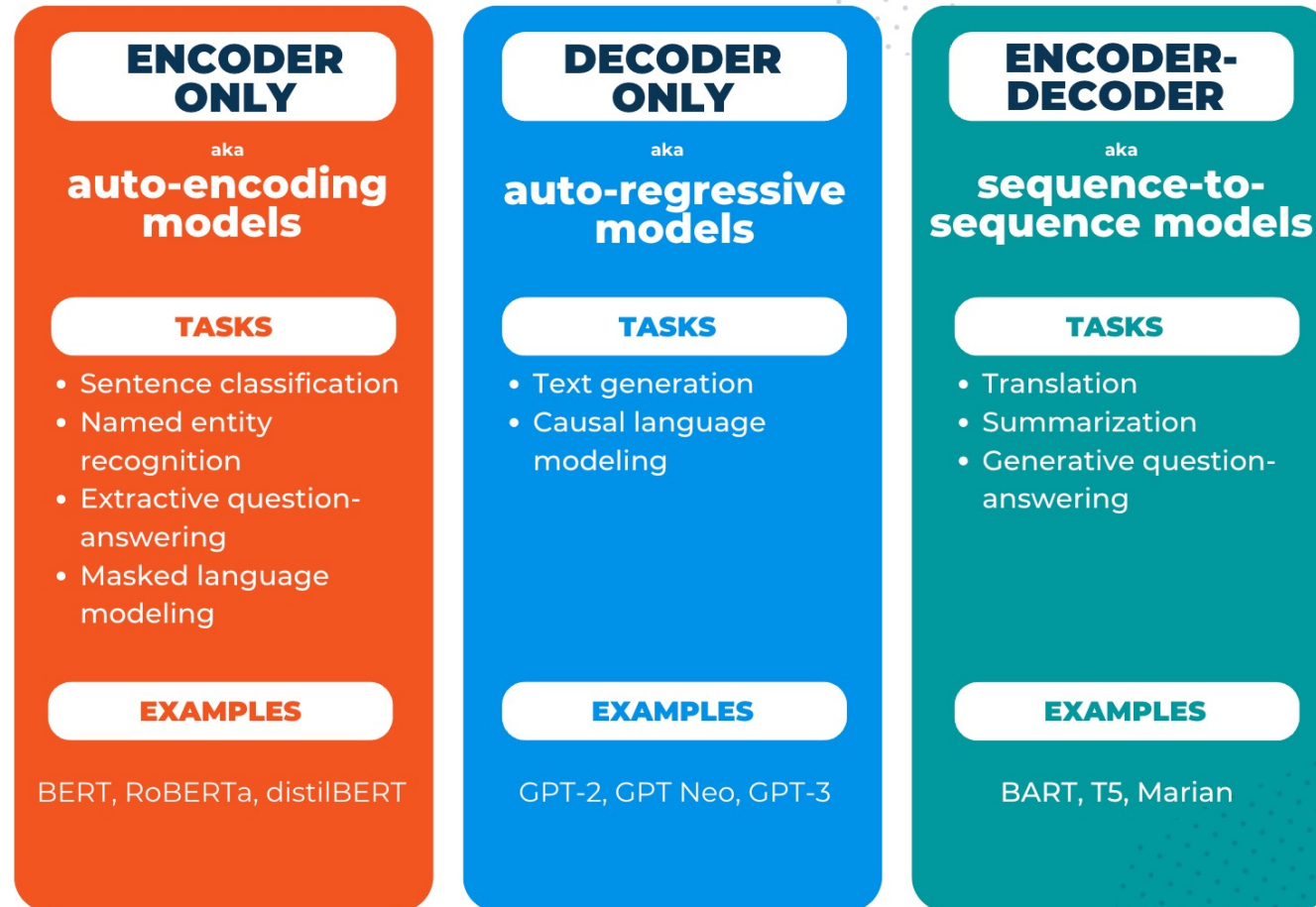
Model can only see previous timesteps

Model is auto-regressive with previous timesteps' outputs

Can also be adapted to generate hidden states by looking before token outputs

Generation

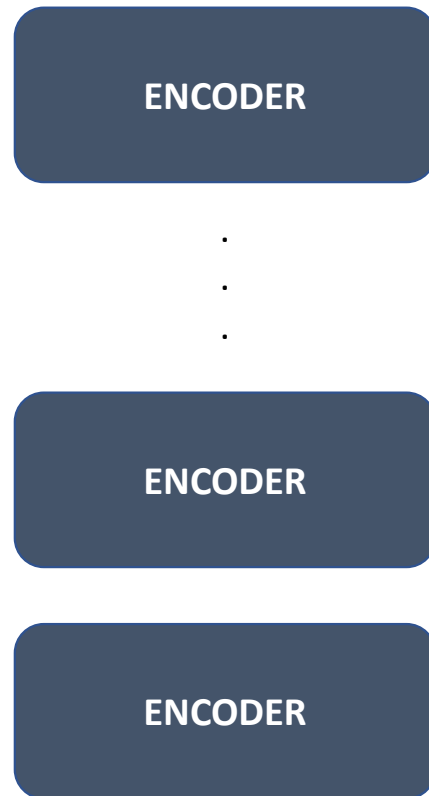
Transformers



Encoder (Nx)

Encoders

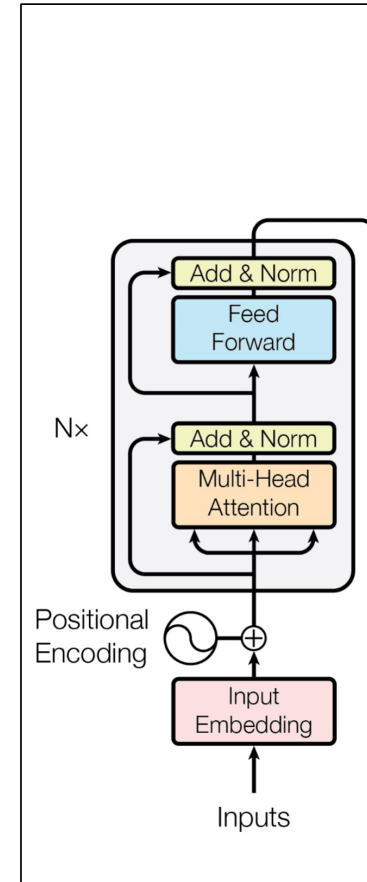
Encoder



Input to Encoder_{i+1}

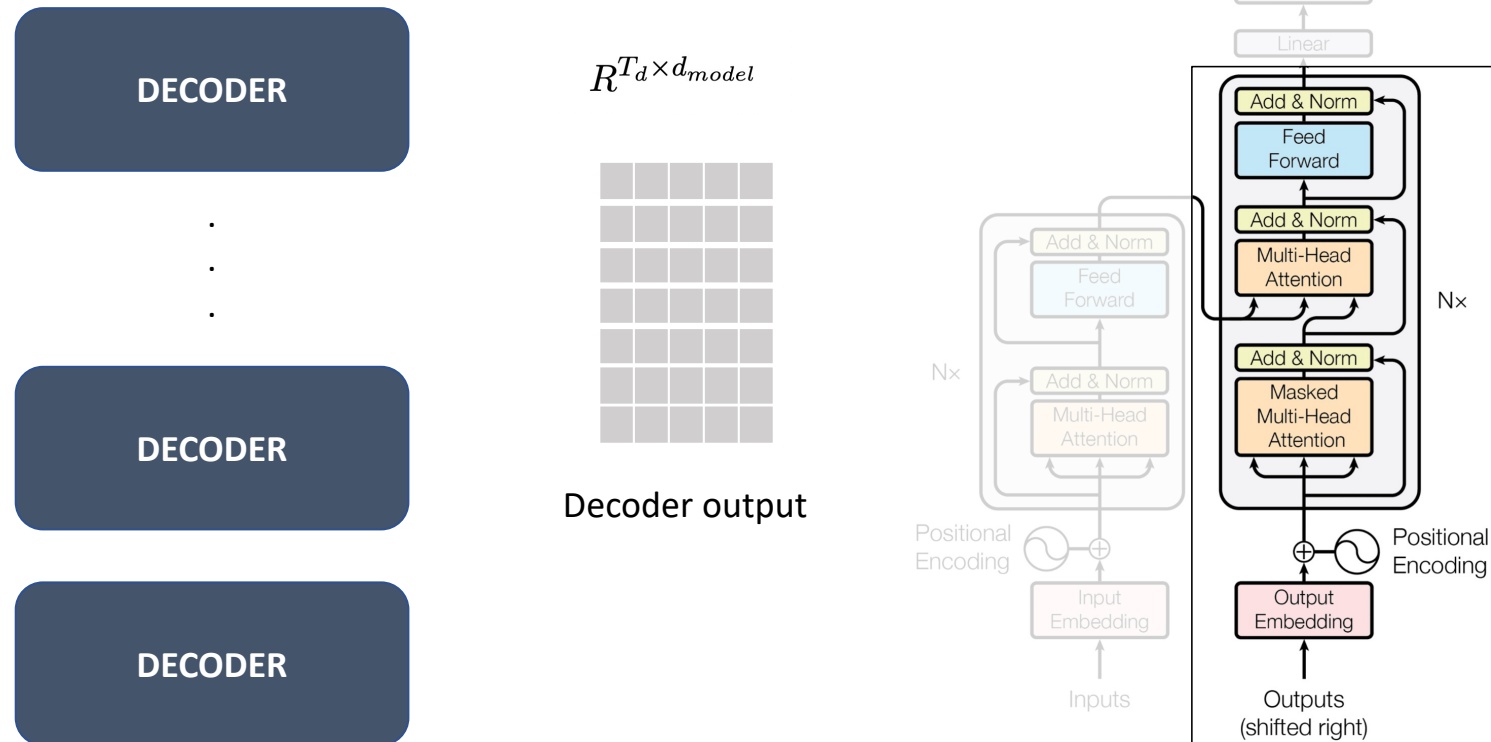
↑

Output from Encoder_i

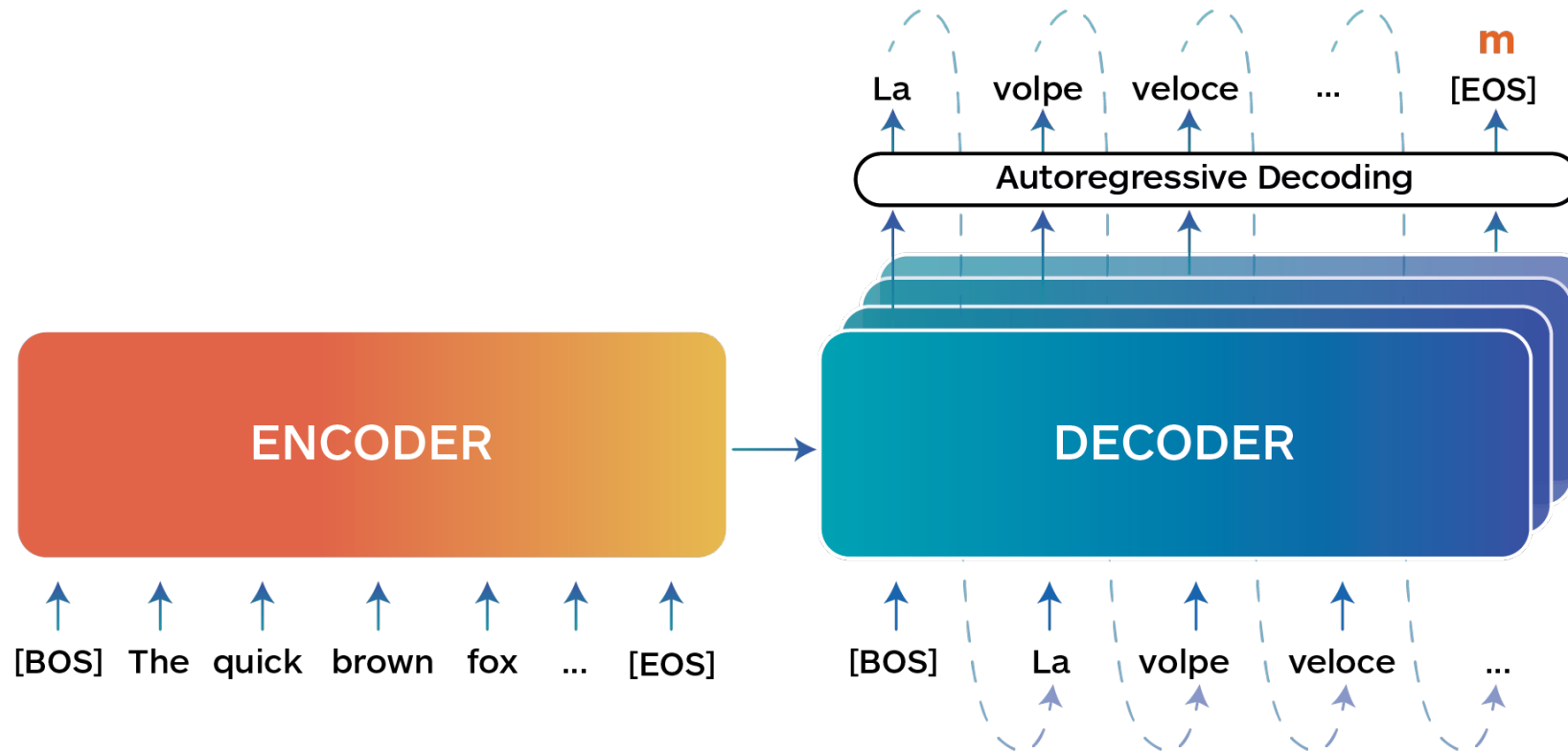


Decoder (Nx)

Decoder



Auto Regressive Decoding



Standard generation procedure in Machine Translation:
Autoregressive Decoding. Each “word” is generated sequentially after the previous one.

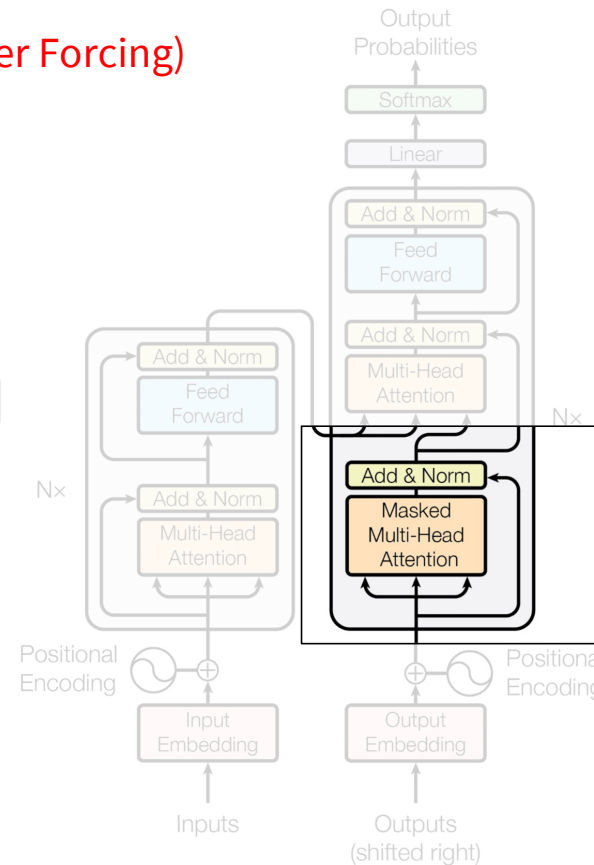
Steps

Masked Multi Head Attention

Decoding step by step (using Teacher Forcing)

1	<sos>	- ∞	- ∞	- ∞	- ∞	- ∞
2	<sos>	Ich	- ∞	- ∞	- ∞	- ∞
3	<sos>	Ich	have	- ∞	- ∞	- ∞
4	<sos>	Ich	have	einen	- ∞	- ∞
5	<sos>	Ich	have	einen	apfel	- ∞
6	<sos>	Ich	have	einen	apfel	gegessen
7	<sos>	Ich	have	einen	apfel	gegessen

Softmax -> - ∞ -> 0



Steps

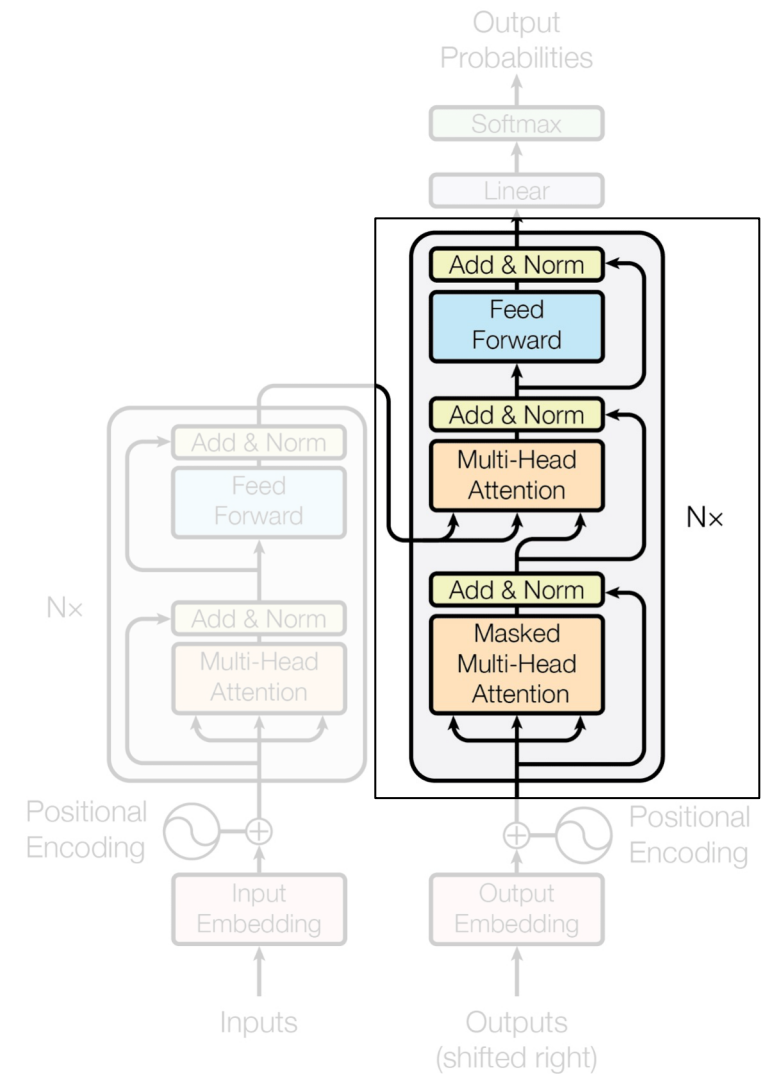
Encoder

Keys from **Encoder Outputs**
Values from **Encoder Outputs**

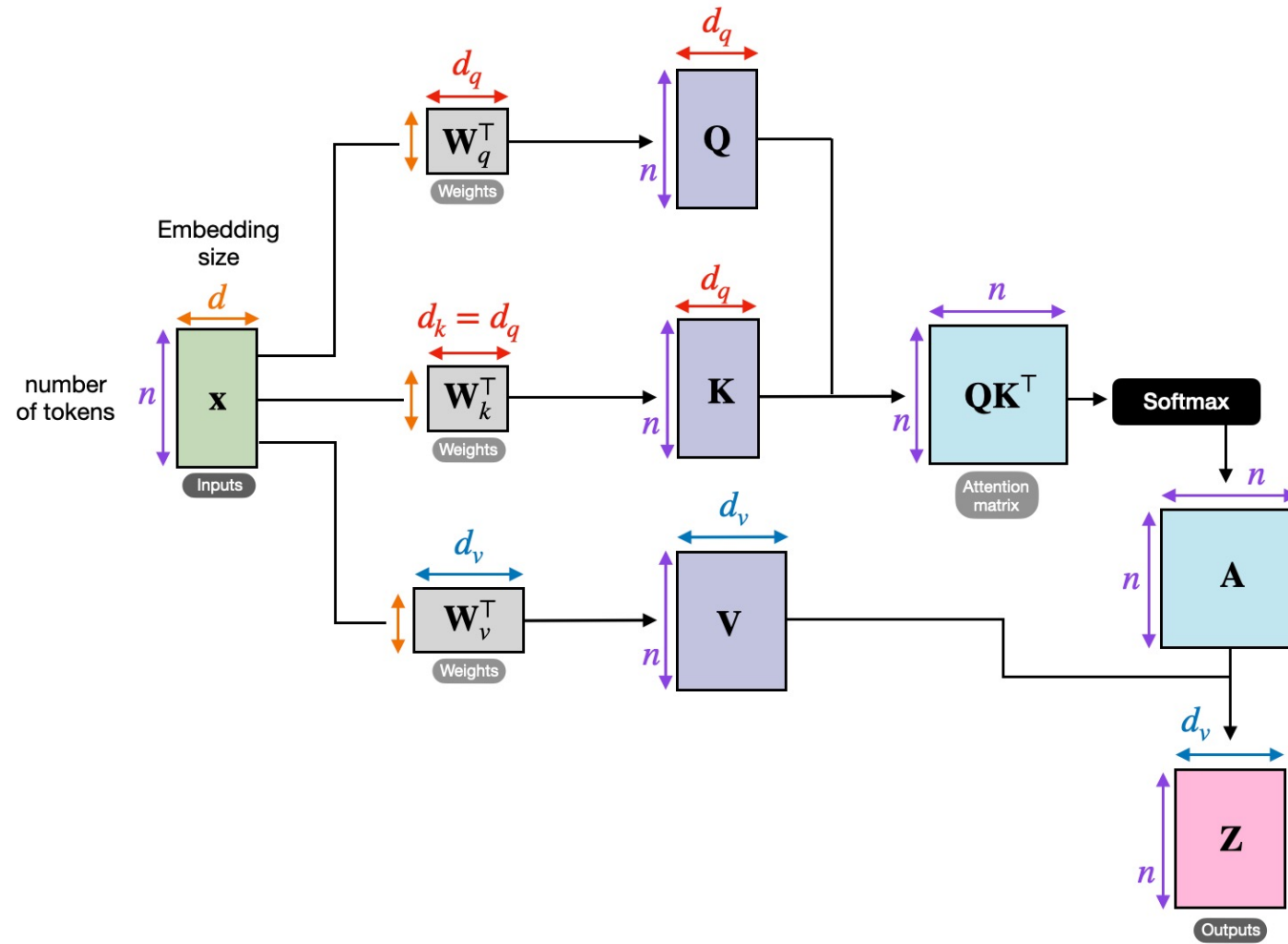
Decoder

Queries from **Decoder Inputs**

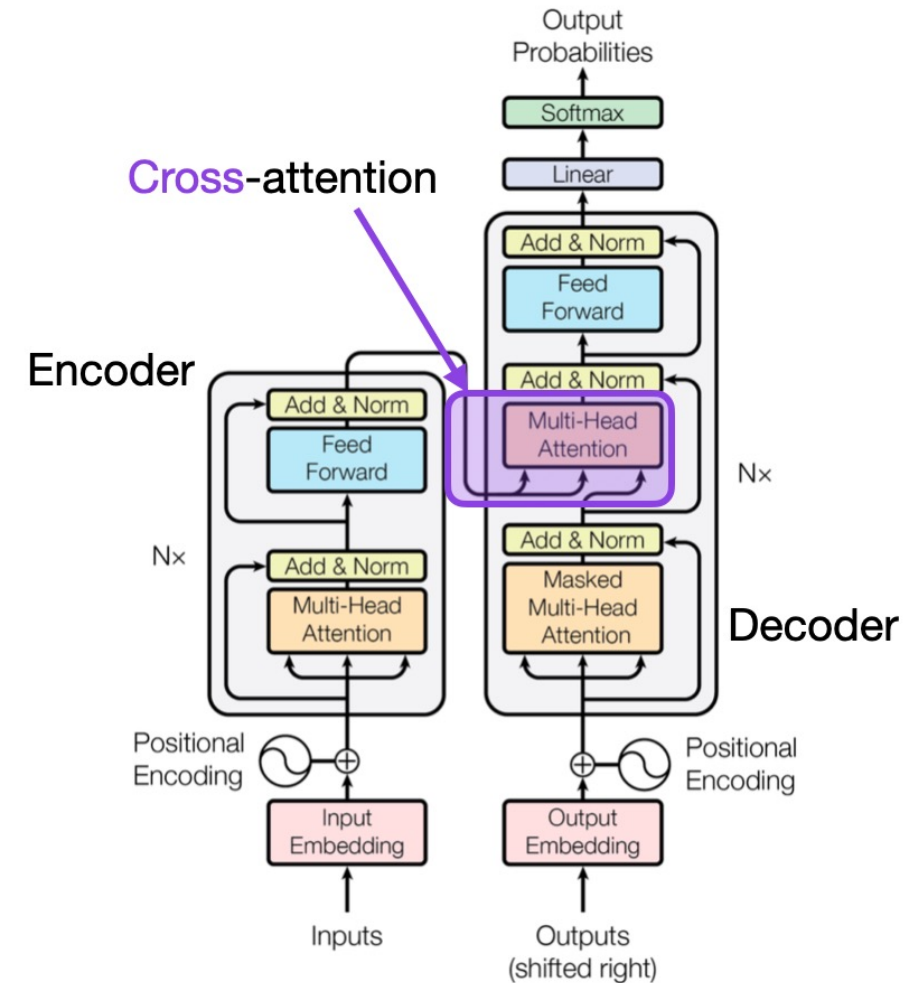
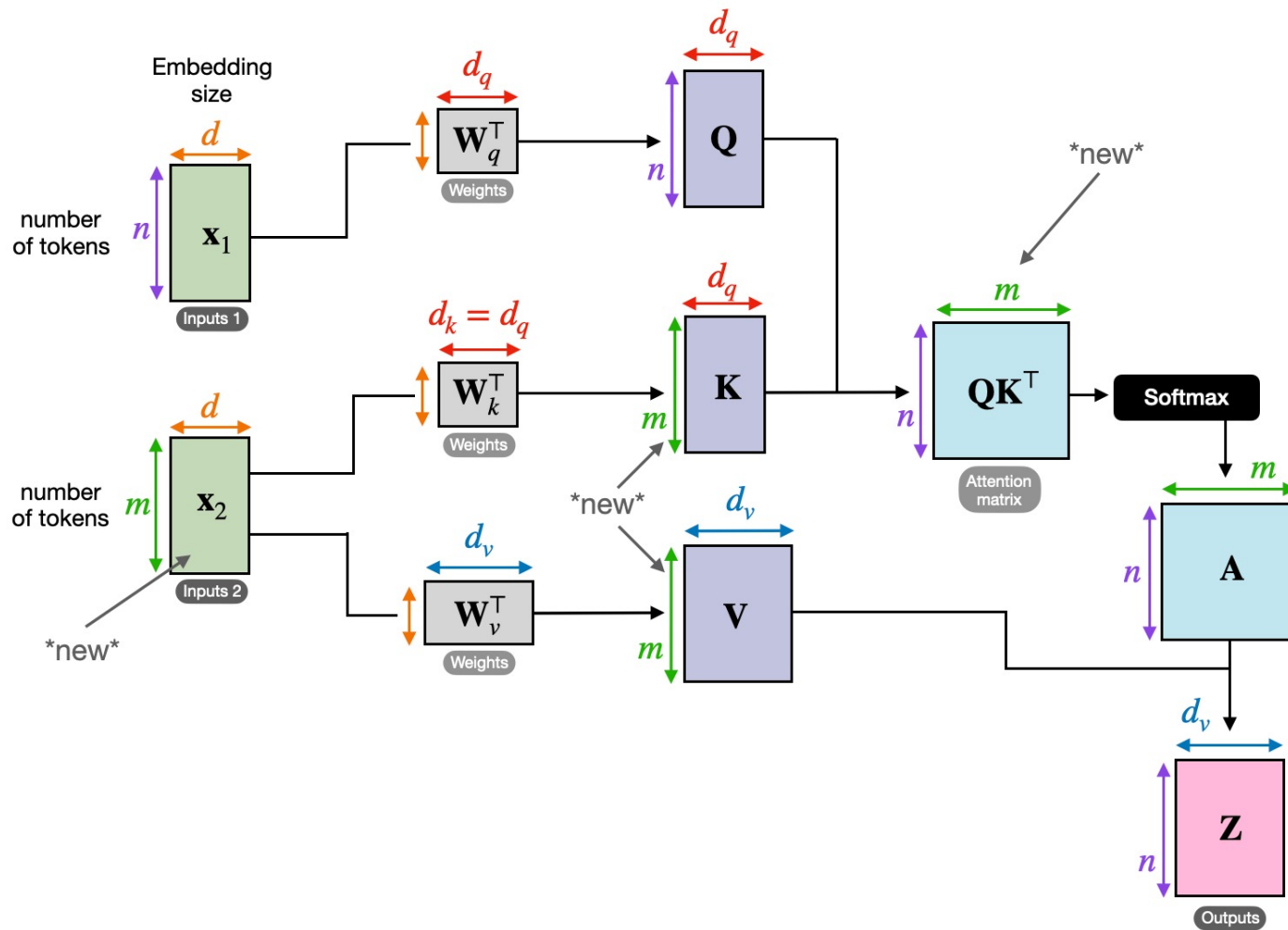
NOTE: Every decoder block receives the same FINAL encoder output



Self Attention



Cross Attention

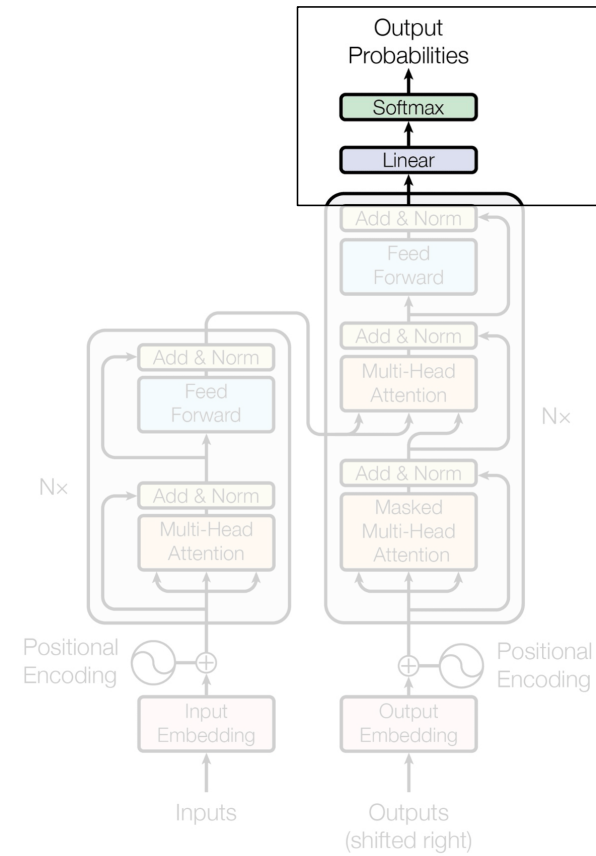
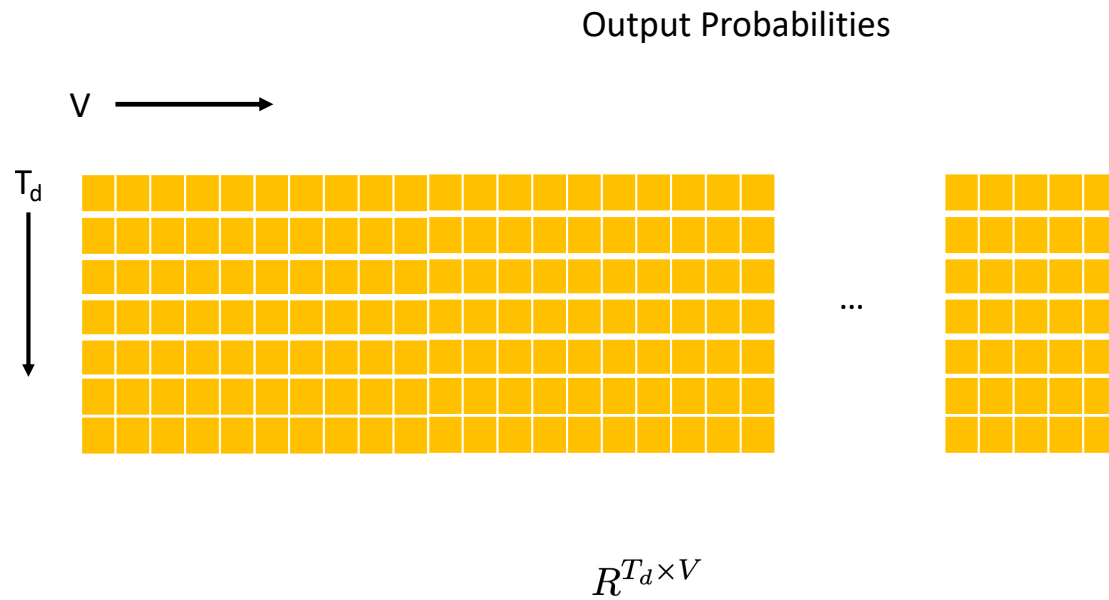


Source: "Attention Is All You Need" (<https://arxiv.org/abs/1706.03762>)

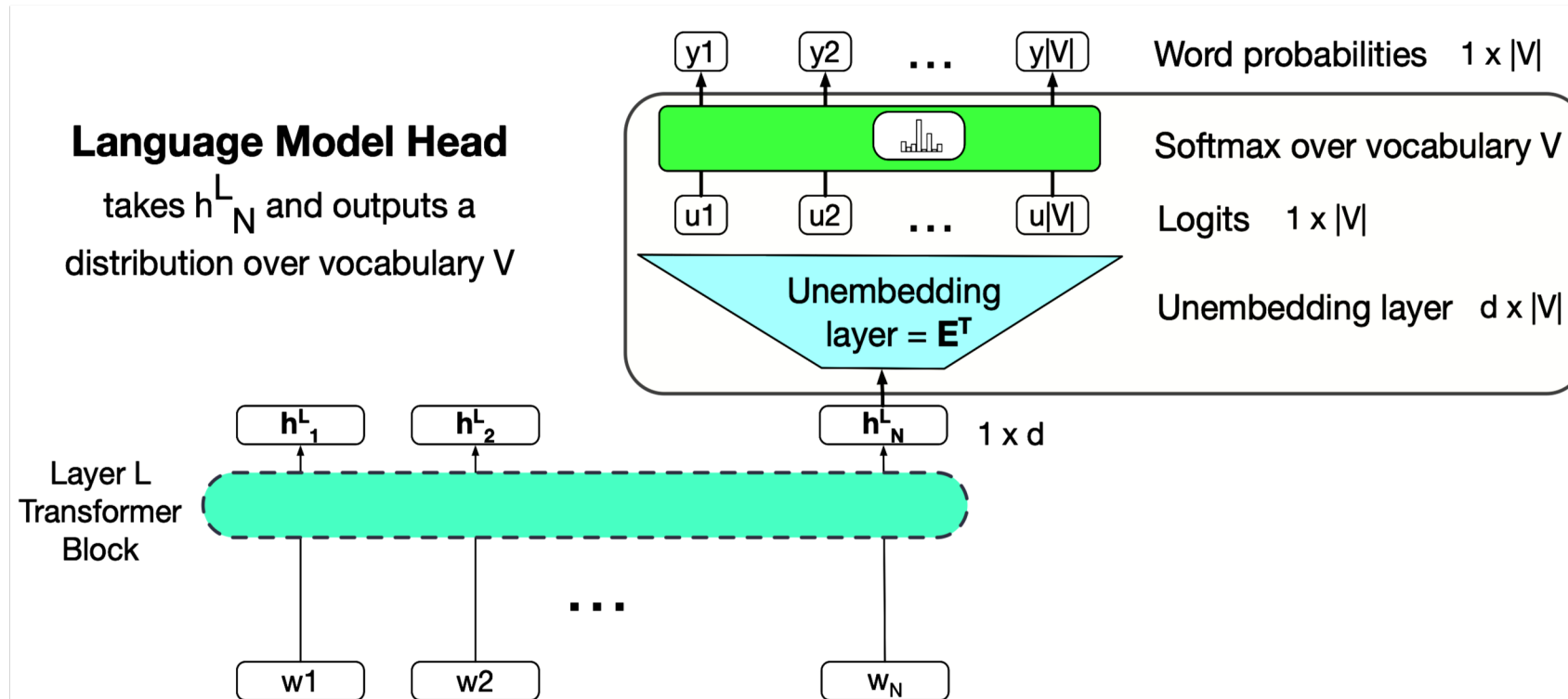
Example: Note that the queries usually come from the decoder, and the keys and values usually come from the encoder.

Steps

Softmax



Language Modelling Head



$$\mathbf{u} = \mathbf{h}_N^L \mathbf{E}^T$$
$$\mathbf{y} = \text{softmax}(\mathbf{u})$$

Transformer

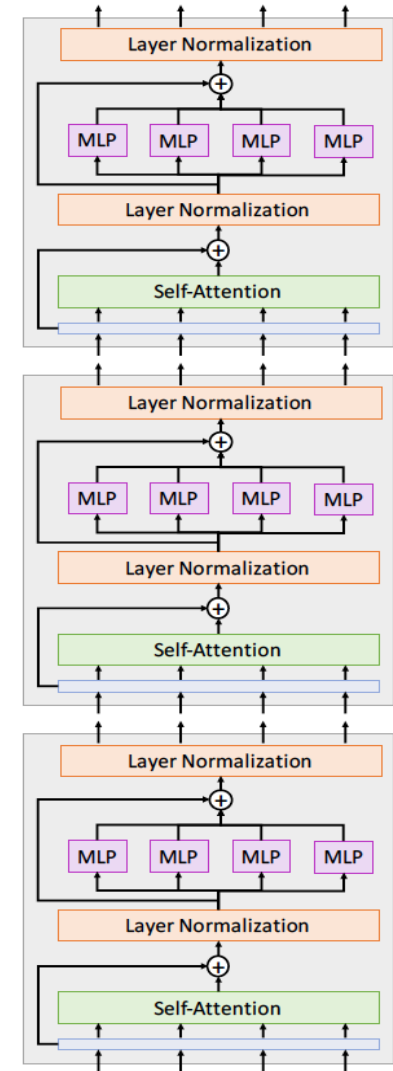
“ImageNet Moment for Natural Language Processing”

Pretraining:

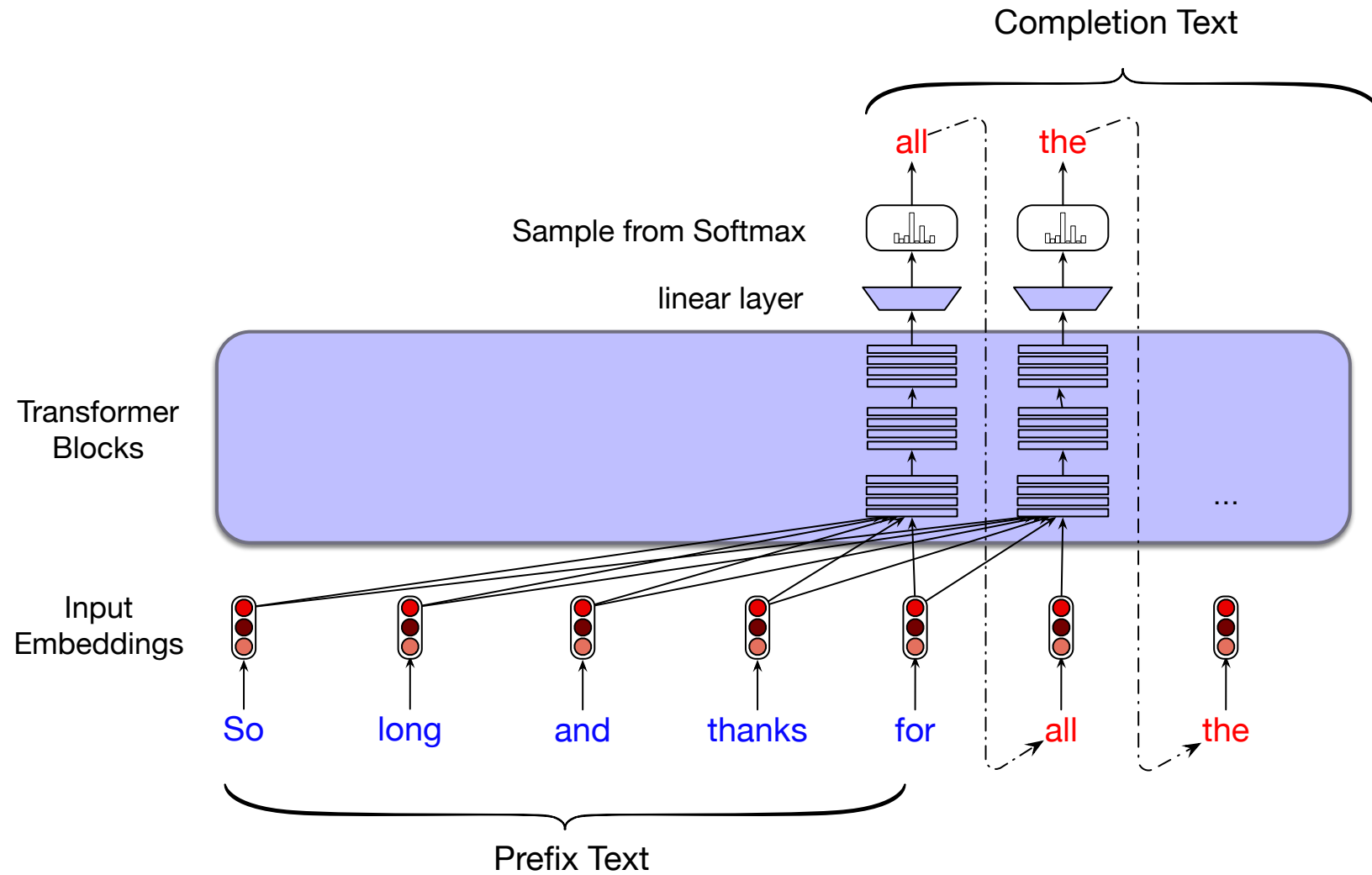
- Download a lot of text from the internet
- Train a giant Transformer model for language modeling

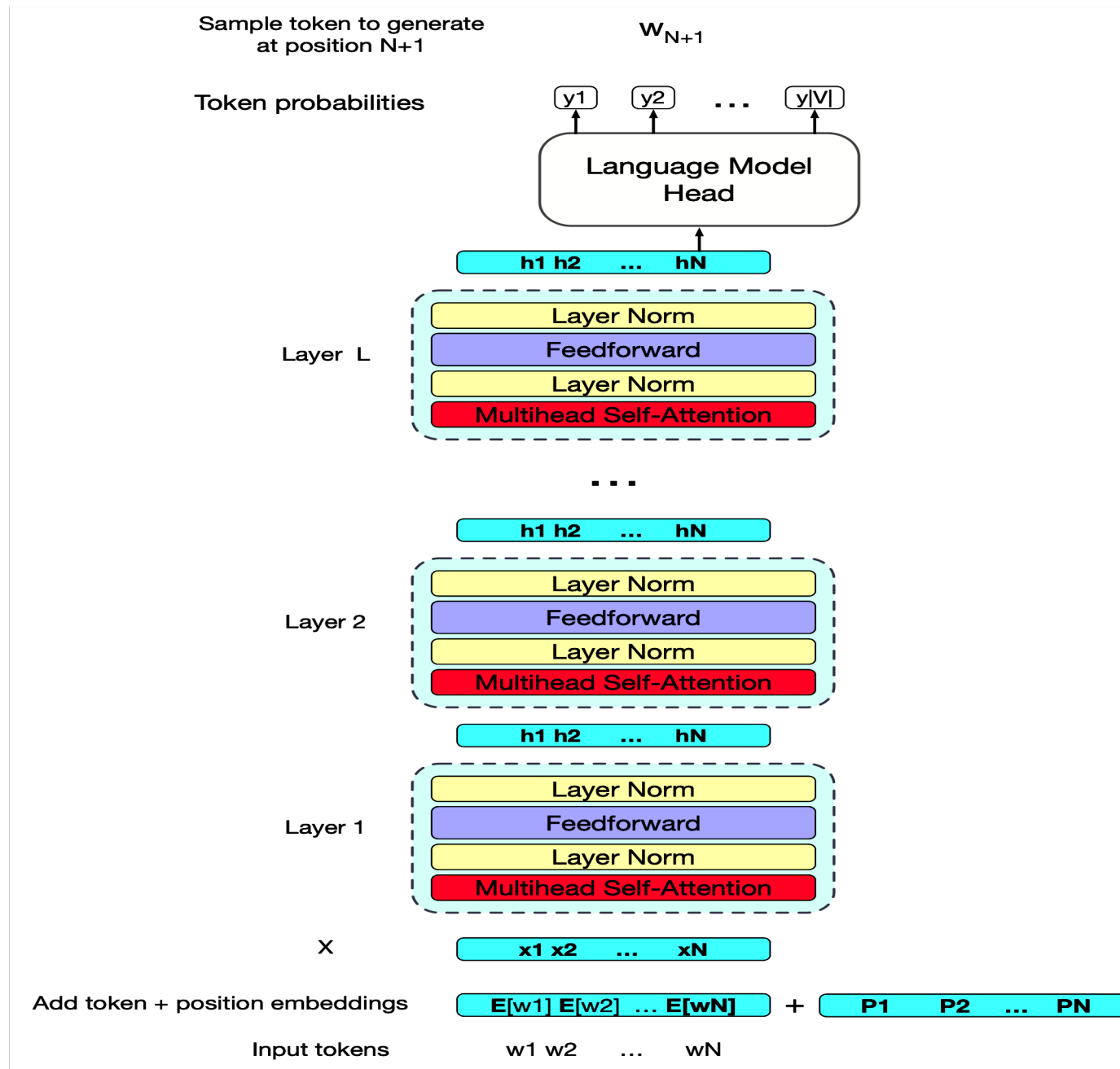
Finetuning:

- Fine-tune the Transformer on your own NLP task



Sentence Completion



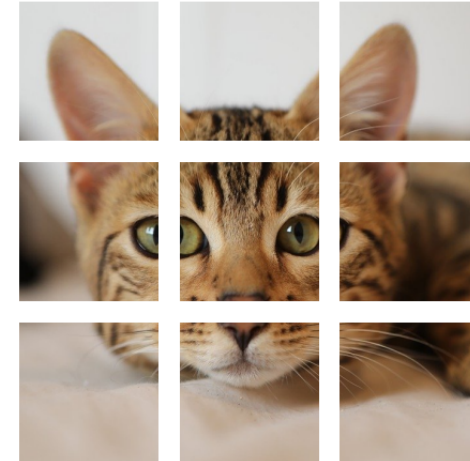


Eg. A Complete Decoder Model

Image to Tokens

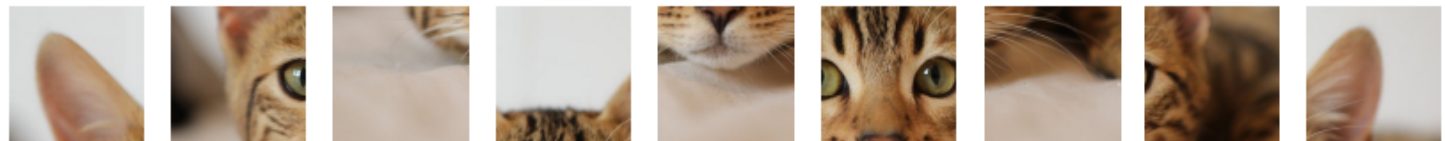


224 X 224



Divide the image into 196 (14×14)
small images of size 16×16

N input patches, each of
shape $3 \times 16 \times 16$



Vision Transformer

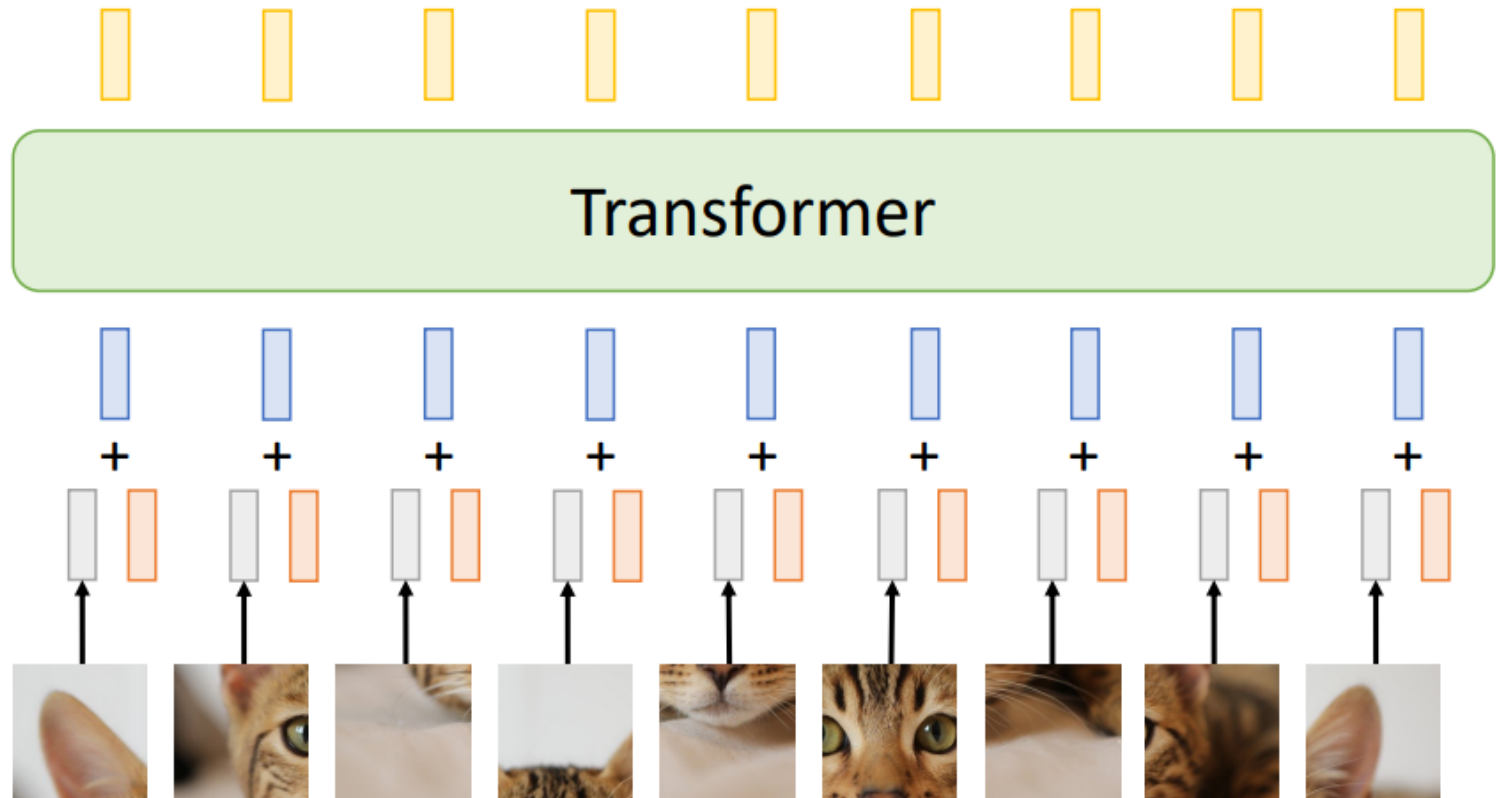
Output vectors

Exact same as NLP
Transformer!

Add positional embedding:
learned D-dim vector

Linear projection to
D-dimensional vector

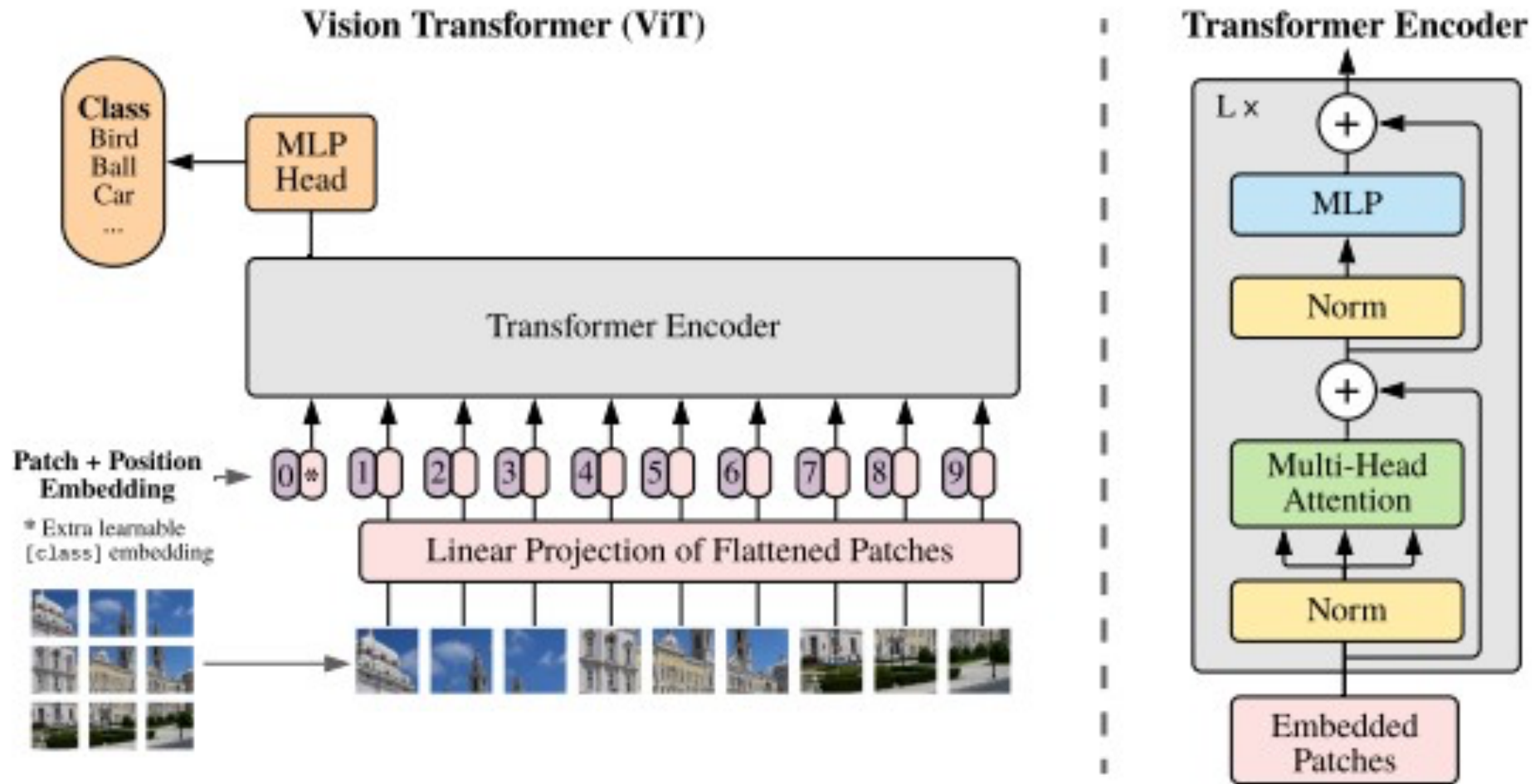
N input patches, each of
shape 3x16x16



Basic Steps

1. Split an image into patches
2. Flatten the patches
3. Produce lower-dimensional linear embeddings from the flattened patches
4. Add positional embeddings
5. Feed the sequence as an input to a standard transformer encoder
6. Pretrain the model with image labels (fully supervised on a huge dataset)
7. Finetune on the downstream dataset for image classification

Vision Transformer



Architecture resembles the one used in language; 'An image is worth 16X16 words, 2020

Questions?